

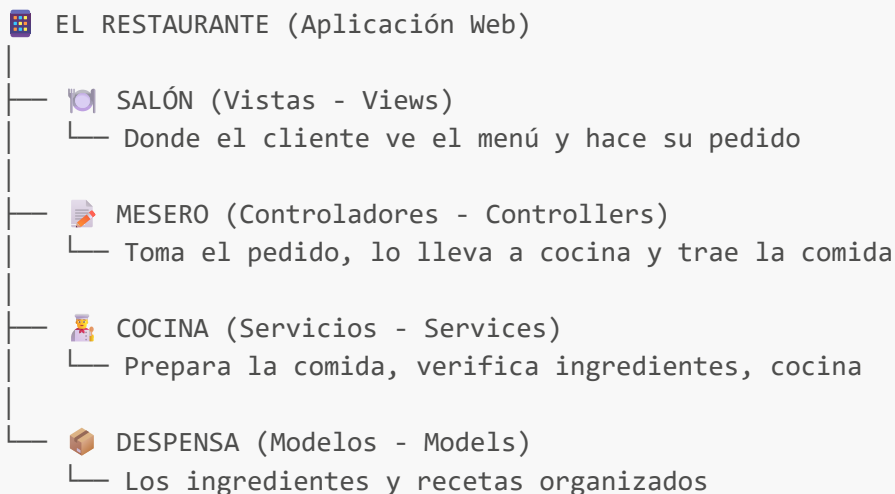
MEMORIA DESCRIPTIVA - ARQUITECTURA DEL PROYECTO "QUIZ VIRTUAL"

🎯 ¿QUÉ ES ESTE PROYECTO?

Este es un sistema web para que los estudiantes respondan un quiz de dos preguntas y los docentes puedan gestionar los resultados. La aplicación guarda la información en internet usando un servicio llamado **MockAPI**.

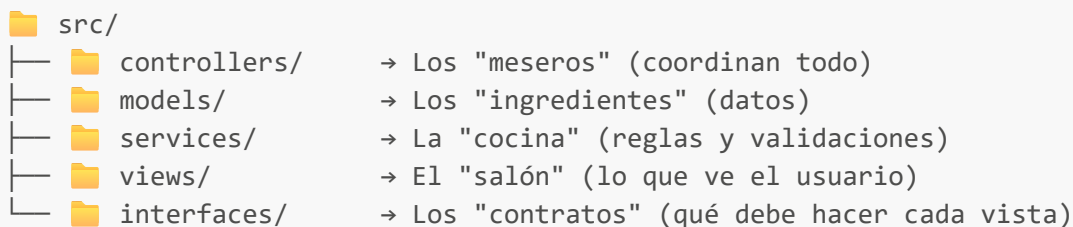
🏠 ARQUITECTURA GENERAL

Imagina que el proyecto es como un **restaurante**:



Cada parte tiene una función específica y **no se mezclan**.

📁 ESTRUCTURA DE CARPETAS



🔄 FLUJO DE TRABAJO (EJEMPLO: AGREGAR ESTUDIANTE)

Paso 1: El usuario llena el formulario

-  VISTA (Cl_vEstudiantes.ts)
 - Muestra campos: cédula, nombre
 - Botón "Agregar"
 - Tabla con lista de estudiantes

Paso 2: El controlador recibe la acción

-  CONTROLADOR (Cl_cEstudiantes.ts)
 - Escucha clic en "Agregar"
 - Crea un objeto estudiante con los datos
 - Llama al servicio

Paso 3: El servicio valida y procesa

-  SERVICIO (Cl_sEstudiantes.ts)
 -  Verifica que cédula sea positiva
 -  Verifica que nombre no esté vacío
 -  Consulta si ya existe ese estudiante
 -  Si todo está bien, guarda en MockAPI
 -  Devuelve mensaje de éxito o error

Paso 4: El controlador actualiza la vista

-  CONTROLADOR → VISTA
 - Muestra mensaje al usuario
 - Si tuvo éxito, recarga la lista

COMPONENTES PRINCIPALES

1. VISTAS (Views) - Cl_v*.ts

¿Qué hacen?

- Muestran la interfaz al usuario
- Capturan los eventos (clics, escritura)
- No toman decisiones, solo muestran y capturan

Ejemplo: Cl_vEstudiantes.ts

```
// Solo captura lo que el usuario escribe
get cedula(): number {
  return parseInt(this.inCedula.value.trim()) || 0;
```

```
}

// Solo muestra la lista en la tabla
mostrarEstudiantes(estudiantes: Cl_mEstudiante[]): void {
  this.tblRegistros.innerHTML = "";
  estudiantes.forEach((estudiante) => {
    this.tblRegistros.innerHTML += `<tr>
      <td>${estudiante.cedula}</td>
      <td>${estudiante.nombre}</td>
    </tr>`;
  });
}
```

2. CONTROLADORES (Controllers) - Cl_c*.ts

¿Qué hacen?

- Coordinan entre vistas y servicios
- Crean los modelos con datos de las vistas
- Llamam a los servicios y muestran resultados
- **NO validan reglas de negocio** (eso es del servicio)

Ejemplo: Cl_cEstudiantes.ts

```
private async onAgregar() {
  // 1. Crear modelo con datos de la vista
  const estudiante = new Cl_mEstudiante({
    cedula: this.vista.cedula,
    nombre: this.vista.nombre,
  });

  // 2. Llamar al servicio (que hace todo el trabajo pesado)
  const resultado = await sEstudiantes.agregar(estudiante);

  // 3. Mostrar resultado
  alert(resultado.mensaje);
  if (resultado.ok) this.cargarEstudiantes();
}
```

3. SERVICIOS (Services) - Cl_s*.ts

¿Qué hacen?

- Contienen TODAS las reglas de negocio
- Validan los datos antes de guardar
- Se comunican con MockAPI
- Devuelven resultados con mensajes claros

Ejemplo: Cl_sEstudiantes.ts

```
static async agregar(nuevoEstudiante: Cl_mEstudiante) {  
  // ✅ VALIDACIÓN 1: Cédula positiva  
  if (nuevoEstudiante.cedula <= 0) {  
    return { ok: false, mensaje: "La cédula debe ser positiva" };  
  }  
  
  // ✅ VALIDACIÓN 2: Nombre obligatorio  
  if (!nuevoEstudiante.nombre) {  
    return { ok: false, mensaje: "El nombre es obligatorio" };  
  }  
  
  // ✅ VALIDACIÓN 3: Verificar si ya existe  
  const chkExiste = await this.existe(nuevoEstudiante.cedula);  
  if (chkExiste.existe) {  
    return { ok: false, mensaje: "Ya existe con esa cédula" };  
  }  
  
  // ✅ Si todo está bien, guardar  
  return super.agregar(nuevoEstudiante.toJSON());  
}
```

4. MODELOS (Models) - Cl_m*.ts

¿Qué hacen?

- Representan la estructura de los datos
- Tienen métodos básicos para convertir a JSON
- Pueden tener lógica simple (ej: `esCorrecto()` en Quiz)

Ejemplo: Cl_mEstudiante.ts

```
export default class Cl_mEstudiante {  
  private cedula: number;  
  private nombre: string;  
  
  constructor({ cedula, nombre }: { cedula: number; nombre: string }) {  
    this.cedula = cedula;  
    this.nombre = nombre;  
  }  
  
  // Convierte el objeto a JSON para enviar a MockAPI  
  toJSON() {  
    return {  
      tabla: "estudiante",  
      cedula: this.cedula,  
      nombre: this.nombre,  
    };  
  }  
}
```

```
}  
}
```



CONEXIÓN CON MOCKAPI

¿Qué es MockAPI?

Es un servicio gratuito que simula una base de datos en internet. Nuestro proyecto lo usa para guardar estudiantes y quizzes de forma permanente.

Jerarquía de Servicios:

```
Cl_sMockApi (Base - métodos genéricos)  
  ↓ hereda  
Cl_sProyecto (Configura URL del proyecto)  
  ↓ hereda  
Cl_sEstudiantes (Métodos específicos para estudiantes)  
Cl_sQuiz (Métodos específicos para quizzes)  
Cl_sEntregas (Métodos específicos para entregas)
```

Métodos Principales de **Cl_sMockApi**:

```
// 1. Obtener todos los registros de una tabla  
static async getTabla({ tabla }): Promise<{ ok, tabla }>  
  
// 2. Verificar si existe un registro  
static async existeId({ tabla, tablaId, tablaIdName }): Promise<{ ok, existe }>  
  
// 3. Agregar nuevo registro  
static async agregar(registro): Promise<{ ok, mensaje }>  
  
// 4. Modificar registro existente  
static async modificar(tablaId, registro, tablaIdName): Promise<{ ok, mensaje }>  
  
// 5. Eliminar registro  
static async eliminar(tablaId, tabla, tablaIdName): Promise<{ ok, mensaje }>
```



FLUJO COMPLETO DE LA APLICACIÓN

Pantalla 1: Quiz del Estudiante (**index.html**)

1. Estudiante entra a index.html
2. Ve formulario con:
 - Campo cédula

- Campo nombre
- Pregunta 1: ¿5 × 8? (input número)
- Pregunta 2: ¿Azul + Amarillo? (select)
- 3. Hace clic en "Enviar"
- 4. Controlador `Cl\_cQuiz` recibe datos
- 5. Servicio `Cl\_sQuiz` valida y guarda
- 6. Muestra mensaje de éxito/error

Pantalla 2: Panel del Docente ([curso.html](#))

1. Docente entra a `curso.html`
2. Ve dos botones:
 - "Ver entregas" → Muestra quizzes respondidos
 - "Estudiantes" → Gestiona lista de estudiantes

Opción A: Ver Entregas

1. Clic en "Ver entregas"
2. Controlador `Cl\_cEntregas` carga todos los quizzes
3. Muestra tabla con:
 - Cédula, Nombre, Respuesta 1, Respuesta 2
4. Checkbox "Solo correctos" filtra en el cliente
5. Botón "Recargar" actualiza datos

Opción B: Gestionar Estudiantes

1. Clic en "Estudiantes"
2. Controlador `Cl\_cEstudiantes` carga lista
3. Muestra formulario CRUD:
 - Agregar nuevo estudiante
 - Modificar existente
 - Eliminar estudiante
4. Cada acción pasa por validaciones del servicio

CONCEPTOS CLAVE

1. Separación de Responsabilidades

- **Vista:** Solo muestra y captura eventos
- **Controlador:** Solo coordina
- **Servicio:** Valida y procesa
- **Modelo:** Solo datos

2. Validaciones en el Servicio

```
// ❌ ANTES (en controlador - MAL)
if (cedula <= 0) {
  alert("Cédula inválida");
  return;
}

// ✅ AHORA (en servicio - BIEN)
if (cedula <= 0) {
  return { ok: false, mensaje: "Cédula inválida" };
}
```

3. Respuesta Estandarizada

Todos los servicios devuelven:

```
{
  ok: boolean,           // ¿Tuvo éxito?
  mensaje: string,       // Mensaje para el usuario
  data?: any,            // Datos (si los hay)
  tabla?: any[]          // Lista de registros (si aplica)
}
```

4. Trabajo en Memoria

- Los datos se cargan UNA VEZ desde MockAPI
- Las operaciones (filtrar, ordenar) se hacen en el navegador
- Solo se vuelve a MockAPI para guardar/cambiar/eliminar

Ejemplo: Filtrar quizzes correctos

```
// En el modelo Cl_mQuices
getQuices(soloCorrectos: boolean = false): Cl_mQuiz[] {
  if (soloCorrectos) {
    // Filtra en el navegador, NO en MockAPI
    return this.quices.filter(quiz => quiz.esCorrecto());
  }
  return this.quices;
}
```

INTERFACES (Contratos)

Las interfaces definen **qué debe hacer** una vista, sin importar **cómo lo haga**.

Ejemplo: I_vEstudiantes.ts

```
export default interface I_vEstudiantes {
  cedula: number; // Propiedad
  nombre: string; // Propiedad
  onAgregar(callback: () => void): void; // Evento
  onModificar(callback: () => void): void; // Evento
  onEliminar(callback: () => void): void; // Evento
  onVolver(callback: () => void): void; // Evento
  mostrar(): void; // Método
  ocultar(): void; // Método
  mostrarEstudiantes(estudiantes: Cl_mEstante[]): void; // Método
  limpiarInputs(): void; // Método
}
```

Ventaja: El controlador no depende de la implementación HTML, solo del contrato.



NAVEGACIÓN ENTRE PANTALLAS

El proyecto usa un sistema simple de **mostrar/ocultar** secciones HTML:

```
<!-- curso.html -->
<section id="curso"> <!-- Menú principal -->
  <button id="curso_btVerEntregas">Ver entregas</button>
  <button id="curso_btEstudiantes">Estudiantes</button>
</section>

<section id="entregas" hidden> <!-- Oculta por defecto -->
  <!-- Contenido de entregas -->
</section>

<section id="estudiantes" hidden> <!-- Oculta por defecto -->
  <!-- Contenido de estudiantes -->
</section>
```

Flujo:

1. Usuario hace clic en "Ver entregas"
2. Controlador `Cl_cCurso` crea instancia de `Cl_cEntregas`
3. `Cl_cEntregas` muestra la sección `#entregas`
4. Cuando termina, oculta `#entregas` y devuelve el control



EJEMPLOS PRÁCTICOS

Ejemplo 1: Agregar Estudiante

Paso a paso:

1. Usuario escribe cédula "123" y nombre "Juan"
2. Hace clic en "Agregar"
3. `Cl_cEstudiantes.onAgregar()`:

```
const estudiante = new Cl_mEstudiante({
  cedula: 123,
  nombre: "Juan"
});
const resultado = await sEstudiantes.agregar(estudiante);
```

4. `Cl_sEstudiantes.agregar()`:

```
// Valida cédula positiva ✓
// Valida nombre no vacío ✓
// Verifica no existe ✓
// Guarda en MockAPI ✓
return { ok: true, mensaje: "Registro guardado con ID: 5" };
```

5. Controlador muestra alerta y recarga lista

Ejemplo 2: Filtrar Quizzes Correctos

Paso a paso:

1. Usuario marca checkbox "Solo correctos"
2. `Cl_cEntregas.onChangeSoloCorrectos()`:

```
this.btRecargarOnClick(); // Recarga con filtro
```

3. `Cl_cEntregas.btRecargarOnClick()`:

```
const resultado = await entregas.getEntregas();
this.modelo.setQuices(resultado.tabla);
// Pide al modelo que filtre
this.vista.mostrarQuices(
  this.modelo.getQuices(this.vista.soloCorrectos)
);
```

4. `Cl_mQuices.getQuices(soloCorrectos = true)`:

```
if (soloCorrectos) {
  return this.quices.filter(quiz =>
    quiz.respuesta1 === 40 && quiz.respuesta2 === "verde"
```

```
);  
}  
return this.quices;
```

TECNOLOGÍAS USADAS

Tecnología	Función
TypeScript	Lenguaje con tipos (más seguro que JavaScript)
HTML5	Estructura de las páginas
CSS	Estilos (mínimos, todo inline)
MockAPI	Base de datos en la nube gratuita
Fetch API	Para conectar con MockAPI

DIAGRAMA DE FLUJO GENERAL



MOCKAPI - Base de Datos en la Nube

- Guarda estudiantes
- Guarda quizzes
- Permite CRUD (Crear, Leer, Actualizar, Eliminar)

LECCIONES PARA ESTUDIANTES

1. Por qué separar en capas?

- **Mantenimiento:** Cambiar una validación solo afecta al servicio
- **Testing:** Puedes probar cada capa independientemente
- **Reutilización:** El mismo servicio puede usarse desde diferentes vistas
- **Claridad:** Cada archivo tiene una responsabilidad clara

2. Por qué validaciones en el servicio?

```
// Si validas en el controlador:  
// - Tienes que repetir la validación en cada lugar que uses el servicio  
// - Si cambias la regla, tienes que buscar en todos los controladores  
  
// Si validas en el servicio:  
// - La validación está en un solo lugar  
// - Todos los que usen el servicio obtienen el mismo comportamiento  
// - Es más fácil de probar
```

3. Por qué interfaces para las vistas?

```
// El controlador depende de la INTERFAZ, no de la implementación  
constructor(vista: I_vEstudiantes) {  
  // No le importa si es HTML, React, Angular, etc.  
  // Solo le importa que cumpla el contrato I_vEstudiantes  
}
```

4. Por qué modelos separados de servicios?

- **Modelo:** Solo representa datos (ej: un estudiante tiene cedula y nombre)
- **Servicio:** Sabe cómo guardar/validar esos datos
- **Separación:** Los datos no saben cómo persistirse

PREGUNTAS FRECUENTES

¿Por qué no validar en el controlador?

Porque el controlador es solo un puente. Si validas ahí, tendrás que repetir la validación en cada controlador que use el mismo servicio.

¿Por qué los servicios son estáticos?

Para que puedan usarse sin necesidad de crear instancias. Es más simple para este proyecto.

¿Por qué MockAPI y no una base de datos real?

Porque es gratuito, fácil de usar, y perfecto para proyectos educativos. No requiere configuración de servidor.

¿Qué pasa si MockAPI se cae?

El servicio devuelve `{ ok: false, mensaje: "Error de conexión" }` y el controlador muestra una alerta al usuario.

¿Se puede usar esta arquitectura con React/Angular?

¡Sí! La arquitectura MVC es independiente del framework. Solo cambiaría la implementación de las vistas.



RESUMEN FINAL

Este proyecto demuestra cómo construir una aplicación web organizada usando:

1. **MVC (Model-View-Controller)** - Separación clásica de responsabilidades
2. **Servicios inteligentes** - Toda la lógica de negocio centralizada
3. **Controladores delgados** - Solo coordinan, no validan
4. **MockAPI** - Almacenamiento permanente en la nube
5. **TypeScript** - Código tipado y seguro

La clave del éxito es: "**Cada cosa en su lugar, y un lugar para cada cosa**".

Elaborado para: Curso de Programación con POO y TypeScript

Proyecto: Quiz Virtual

Fecha: Mayo 2026

Versión: 1.0 (Después de refactorización)